

Dividers

MDE - Advanced Engineering Topics, ITESO

Gomez Enriquez, Gerardo

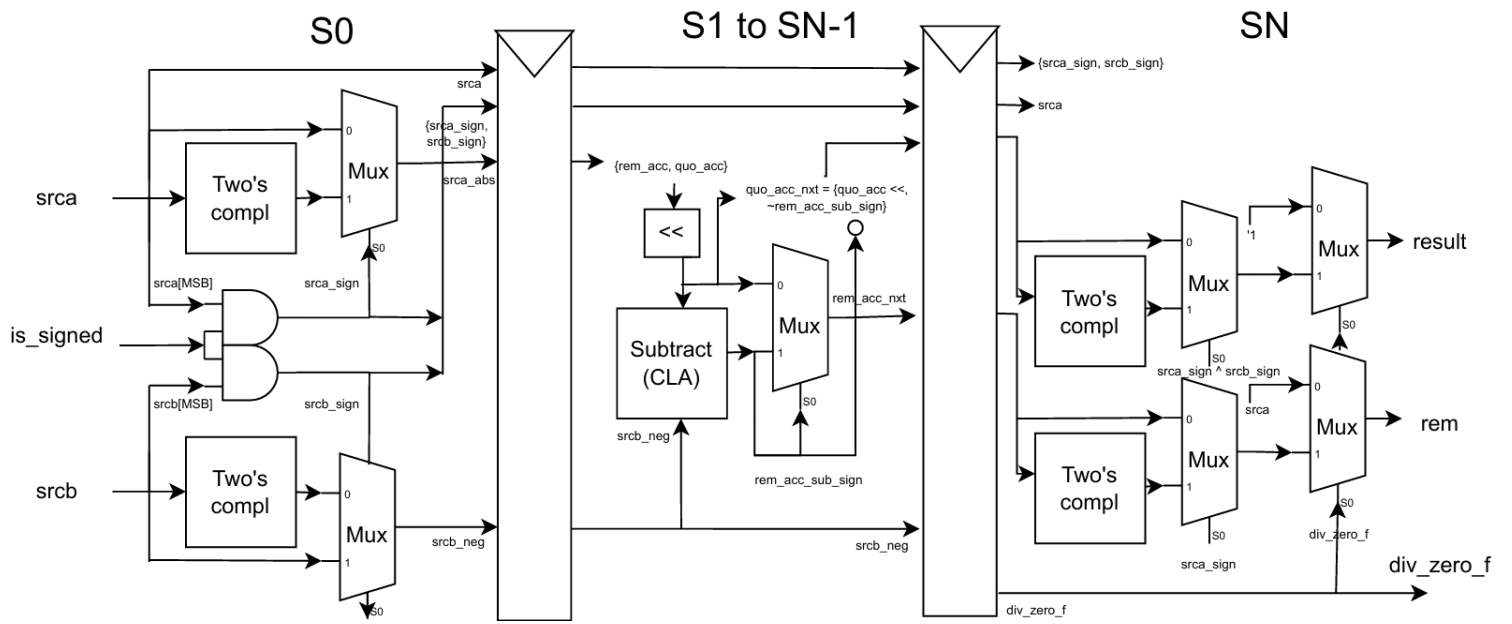
gerardo.gomez@iteso.mx

Guadalajara, Mexico

Architecture and considerations

This is a pipelined 64-bit divider based on the restoring division algorithm. To optimize the pipeline for frequency, each pipe stage handles an addition/subtraction operation plus miscellaneous logic such as multiplexors, shifters by constant amount or qualifications. These architectural choices resulted in a 66-stage pipeline described below.

Stage 0	Stage 1	Stage 2 to N-2 (63)	Stage N-1 (64)	Stage N (65)
Extract operands signs and qualify with is_signed	Initialize AQ		Detect divide-by-zero	Get quotient two's complement
Get dividend two's complement	Shift AQ	Shift AQ	Shift AQ	Get remainder two's complement
Get divider two's complement	Subtract with divider	Subtract with divider	Subtract with divider	Apply quotient sign
Select dividend absolute value	Select partial reminder (subtracted or "restored")	Select partial reminder (subtracted or "restored")	Select partial reminder (subtracted or "restored")	Apply remainder sign
Select divider with negative sign (for subtractions)	Issue quotient bit	Issue quotient bit	Issue quotient bit	Override outputs if divide-by-zero



Implementation details

Summary

Metric	Value
LUTs (all normal mode)	20,646
FFs (pipe stages)	16,513
Fmax (Slow 1200 mV 85 °C)	141.0 MHz
Max LUT depth	12

Area breakdown

CLA instances

Adder role	Instance name	Count
Restoring-stage trial subtractor	gen_restoring_div[*].cla_rem_acc_sub	64 (one per stage)
Input two's-complement	cla_srca_comp2, cla_srcb_comp2	2
Output two's-complement (sign correction)	cla_quo_comp2, cla_rem_comp2	2
	Total	68

LUTs per restoring stage

Stages	Subtractor LUTs
Steady-state (stages 3–64)	~247 avg (min 238, max 256)
Stage 2	62
Stage 1	22
Sum of all 64 stage subtractors	15,414

- Stages 1–2 are much smaller because the partial remainder is still mostly zero that early (rem_acc initialized to 0, only 1–2 dividend bits shifted in), so synthesis prunes most of the subtractor.
- The table summarizes only the subtractor. Each stage also has shift + restore-select MUX + quotient-bit logic. A fully-loaded steady-state stage is closer to ~250 (subtract) + ~50–80 (shift/restore) $\approx$ 300+ LUTs.

FFs per restoring stage

Register	Width (bits)
rem_acc_ss	65 (WIDTH+1)
quo_acc_ss	64 (WIDTH)
a_ss (dividend passthrough)	64
b_neg_ss (subtrahend passthrough)	64
a_sign_ss, b_sign_ss	1 + 1
Total per restoring stage	259

- This information is derived analytically from the pipelined registers in each stage.
- Based on this analysis, the total number of FFs only from restoring pipe stages should be $259 \times 64 = 16,576$ which is above the total number reported by Quartus due to registers removed during synthesis due to optimizations (i.e. registers merged with other registers).

Timing breakdown

Critical path

The critical paths are each restoring iteration in the partial remainder computation.

From *rem_acc_ss[52][8]* → To *rem_acc_ss[53][43]*

Logic description	Node	Count
CLA carry look-ahead ($c\sim, c[*]$)	gen_restoring_div[52].cla_rem_acc_sub gen_lvl[*].gen_lcu[*].lcu4 $c\sim*/c[*]$ combout	6
CLA group-generate ($gg\sim$)	... gen_lvl[1].gen_lcu[0].lcu4 $gg\sim 2/gg\sim 3/gg\sim 4$ combout	3
Nested carry-into-carry ($c[3]\sim*$)	... gen_lvl[0].gen_lcu[10].lcu4 $c[3]\sim 2/c[3]\sim 3$ combout	2
Restore/quotient select MUX	... rem_acc_nxt_ss[52][43]~3307 combout	1
	Total	12

This critical path starts from the partial remainder in stage 52, goes through the subtractor (CLA) until the final MUX that checks the subtraction sign to choose the subtracted remainder or the original partial remainder before the subtractor (“restored”).

Other timing parameters

- **Latency:** 66 cycles (Restoring divider processes 1 bit per cycle + 1 cycle to get absolute values + 1 cycle to apply sign to result).
- **Throughput:** 1 operation per cycle / ~64 bits per cycle due to pipeline.
- **Slack:** Significantly negative. Frequency was set to 1 GHz for analysis purposes and getting the maximum possible frequency, which was 141.0 MHz

Conclusion

Pipelining the restoring divider became very expensive because the area cost grows linearly with operands width. The design registers the full $\{rem_acc, quo_acc\}$ datapath plus all sideband signals (a_ss , b_neg_ss , a_sign_ss , b_sign_ss) across all stages. Plus, the algorithm requires one subtractor in each restoring stage. This is a typical cost in area of a fully pipelined bit-per-cycle divider.

This implementation is not suitable for high performance 64-bit CPUs due to the high latency. The penalty for branch mispredictions would require refilling the 66-stage pipeline every time. Other division algorithms / architectures are more suitable for high-performance CPUs, which reduce area and latency drastically.

The restoring algorithm is more suitable for low-performance/ area-constrained designs when reusing the hardware using an FSM for each iteration — a trade-off between area and throughput acceptable for general-purpose dividers.